



Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: Edwin Santisteban **Carné:** 1047425

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una canalización CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado

Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:

GitHub: <https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet>

El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

Realización de la actividad

1. Preparar el repositorio

En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable

2. Revisar la aplicación:

Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>



3. Crear DotNet Env

- En terminal, instalar: **dotnet add package DotNetEnv**
- Crear .env file en raíz de proyecto, valores ->
AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/
AZURE_STORAGE_CONTAINER_NAME=default
- Crear .env.production con los mismos valores en el mismo lugar
- Añadir cambio brindado en program.cs
- dotnet clean -> dotnet restore -> dotnet build -> dotnet run

4. Crear recursos en Azure

En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.

5. Conectar App Service con Blob Storage

Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

- Azure Portal → **Storage accounts** → **tu Storage Account** → **Data storage** → **Containers**
- Crear contenedor "default"
- Activa identidad administrada en App Service: **Azure Portal** → **App Service** → **Settings** → **Identity** → **System assigned: on** → **Save (web app creada semana 13)**
- Configurar la conexión entre App Service y Blob Storage: **App Service** → **Settings** → **Service Connector** → **Create** → **Seleccionar "Storage Blob"** → **Seleccionar Storage Account [tu_blob_name]** → **Authentication "System-assigned managed identity"** → **Review + Create**.
- Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → **Settings** → **Endpoints** → **Blob service**

6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

- Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: **az --version**
- Inicie sesión en Azure: **az login**
- Verifique la cuenta de Azure activa: **az account show**
- Verifique que pueda obtener un token para Azure Storage: **az account get-access-token --resource <https://storage.azure.com/>**
- Si se muestra un accessToken la autenticación esta correcta, ejecutar -> **dotnet run**
- Si hay error en acceso, ejecutar en terminal vs code: **az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"**

Obtener info: **az account show --query "{subscriptionId:id, user:user.name}" -o table**

az storage account show --name "tu_storage" --query "{id:id, resourceGroup:resourceGroup}" -o table

- Cerrar process ctr +c y logear de nuevo: **az logout** y **az login**, Por ultimo **dotnet run**



Nota: no olvidar subir cambios a repo github fork

7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo "azure-pipelines.yml"

Colocar en el archivo lo siguiente:

trigger:

- main

pool:

vmImage: 'windows-latest'

variables:

buildConfiguration: 'Release'

dotnetVersion: '9.x'

Completar posteriormente

azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'

appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

1. Instalar .NET 9

- task: UseDotNet@2

displayName: 'Instalar .NET 9 SDK'

inputs:

packageType: 'sdk'

version: '\$(dotnetVersion)'

2. Restaurar dependencias

- task: DotNetCoreCLI@2

displayName: 'Restaurar dependencias'

inputs:

command: 'restore'

projects: '**/*.csproj'

3. Configurar ambiente de producción

- powershell: |

Copy-Item ".env.production" ".env" -Force

displayName: 'Configurar .env de producción'

4. Compilar

- task: DotNetCoreCLI@2

displayName: 'Compilar aplicación'

inputs:

command: 'build'

projects: '**/*.csproj'

arguments: '--configuration \$(buildConfiguration) --no-restore'



5. Publicar

```
- task: DotNetCoreCLI@2
  displayName: 'Publicar aplicación'
  inputs:
    command: 'publish'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-build'
    zipAfterPublish: true
```

6. Desplegar en Azure App Service

```
- task: AzureWebApp@1
  displayName: 'Desplegar en Azure App Service'
  inputs:
    azureSubscription: '$(azureSubscription)'
    appType: 'webApp'
    appName: '$(appName)'
    package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```

8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

- Conectarse a Azure DevOps con misma cuenta que azure portal -> <https://aex.dev.azure.com/me>
- Crear pipeline: **En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml**
- Probar seleccionando "run"
- Conectar Azure DevOps con Azure: **Project Settings -> Service connections -> New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre "MIA_Connection" -> Seleccionar "grant access permission to all pipelines" -> Save**

8. Conectar Pipeline con App

- Obtener nombre app: En Azure Portal App Services -> tu aplicación -> Copia nombre app
- en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

```
azureSubscription: TU_Service_Connection_name
appName: Tu_app_name
```

- Al hacer commit, verificar el pipeline se ejecuta automáticamente



9. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

- acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings
-> colocar `AZURE_STORAGEBLOB_RESOURCEENDPOINT`
https://tu_storage.blob.core.windows.net/
(si existe obviar)
- Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica
- En la app publicada realizar cambios en archivos
- Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios

10. Preguntas de analisis

- ¿Para qué sirve el archivo `.env` en la aplicación?
Guarda la configuración (endpoint y contenedor) fuera del código. La app la lee como variables de entorno, así puedes cambiar de cuenta sin recompilar.
- ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?
El disco de App Service es limitado y se pierde al redesplegar o escalar. Blob Storage es duradero, escalable, compartido entre instancias y separa los datos del código.
- ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?
La URL de App Service (`*.azurewebsites.net`) es la dirección de tu app web. El endpoint de Blob (`*.blob.core.windows.net`) es la dirección del almacenamiento, y la usa tu código con el SDK.
- ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?
La key da acceso total y puede filtrarse. La identidad administrada no tiene secretos que guardar o rotar, usa tokens temporales y permite mínimo privilegio con RBAC.
- ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?
GitHub: guarda el código.
Azure DevOps: ejecuta el pipeline (compilar y desplegar).
Service Connection: permite al pipeline autenticarse en Azure sin secretos en el YAML.
- ¿Qué ocurre después de hacer push a main?
El push activa el pipeline, que restaura, compila y publica. Luego despliega en la Web App mediante la Service Connection, y la app reinicia con la nueva versión.
- Si el pipeline termina correctamente pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?
Es un problema de autorización, no de despliegue. Revisaría:
 - Que la identidad administrada esté habilitada en la Web App.
 - Que tenga el rol *Storage Blob Data Contributor* (no basta Owner/Contributor).
 - Que el rol haya propagado (esperar unos minutos y reiniciar).
 - El firewall de la cuenta de almacenamiento.
 - Que el endpoint y el contenedor sean correctos.
 - El log stream: `AuthorizationPermissionMismatch` indica rol faltante; `AuthorizationFailure` indica red o firewall.



Dentro del repositorio de GitHub, cree la carpeta semana-14/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.

serviceconnector-webapp-storageblob-dotnet / README.md

edsantisteban Update README.md

page_type	sample
languages	csharp
products	azure service-connector
urlFragment	serviceconnector-webapp-storageblob-dotnet

Using Service Connector to connect Azure WebApp with Azure Storage Blob

The repository offers the sample codes of connecting Azure Storage Blob to Azure WebApp with `system managed identity`. Follow the [steps](#) below to create and verify the connection.

EDWIN SANTISTEBAN 1047425

1. Prerequisites

- An Azure account with an active subscription. [Create an account for free.](#)
- Install the [Azure CLI 2.18.0](#) or higher to provision and configure Azure resources.

Se realizó un fork del repo dado, se agregó al README el nombre, apellido y carnet.

https://localhost:5001

An unhandled exception occurred while processing the request.

AggregateException: One or more errors occurred. (Retry failed after 6 tries. Retry settings can be adjusted in ClientOptions.Retry. (Host desconocido. (miarchivos2026.blob.core.windows.net:443)) (Host desconocido. (miarchivos2026.blob.core.windows.net:443)) (Host desconocido. (miarchivos2026.blob.core.windows.net:443)) (Host desconocido. (miarchivos2026.blob.core.windows.net:443)) (Host desconocido. (miarchivos2026.blob.core.windows.net:443)) (Host desconocido. (miarchivos2026.blob.core.windows.net:443)))

System.Threading.Tasks.Task.ThrowIfExceptional(bool includeTaskCanceledExceptions)

RequestFailedException: Host desconocido. (miarchivos2026.blob.core.windows.net:443)

Azure.Core.Pipeline.HttpClientTransport.ProcessAsync(HttpMessage message, bool async)

RequestFailedException: Host desconocido. (miarchivos2026.blob.core.windows.net:443)

Azure.Core.Pipeline.HttpClientTransport.ProcessAsync(HttpMessage message, bool async)

RequestFailedException: Host desconocido. (miarchivos2026.blob.core.windows.net:443)

Azure.Core.Pipeline.HttpClientTransport.ProcessAsync(HttpMessage message, bool async)

RequestFailedException: Host desconocido. (miarchivos2026.blob.core.windows.net:443)

Azure.Core.Pipeline.HttpClientTransport.ProcessAsync(HttpMessage message, bool async)

RequestFailedException: Host desconocido. (miarchivos2026.blob.core.windows.net:443)

Azure.Core.Pipeline.HttpClientTransport.ProcessAsync(HttpMessage message, bool async)

RequestFailedException: Host desconocido. (miarchivos2026.blob.core.windows.net:443)

Azure.Core.Pipeline.HttpClientTransport.ProcessAsync(HttpMessage message, bool async)

Stack Query Cookies Headers Routing

AggregateException: One or more errors occurred. (Retry failed after 6 tries. Retry settings can be adjusted in ClientOptions.Retry. (Host desconocido. (miarchivos2026.blob.core.windows.net:443)) (Host desconocido. (miarchivos2026.blob.core.windows.net:443)) (Host desconocido. (miarchivos2026.blob.core.windows.net:443)) (Host desconocido. (miarchivos2026.blob.core.windows.net:443)) (Host desconocido. (miarchivos2026.blob.core.windows.net:443)) (Host desconocido. (miarchivos2026.blob.core.windows.net:443)))

System.Threading.Tasks.Task.ThrowIfExceptional(bool includeTaskCanceledExceptions)

System.Threading.Tasks.Task.Wait(int millisecondsTimeout, CancellationToken cancellationToken)

System.Threading.Tasks.Task.Wait()

WebStorageSample.Pages.IndexModel.OnGet() in Index.cshtml.cs

Request fallido, no reconoce miaarchivos2026.



Microsoft Azure portal showing the configuration of a managed identity for a web app. The page title is "miaazurestorageES | Identity". The left sidebar shows the navigation menu with "Identity" selected. The main content area shows the "System assigned" tab for the managed identity. The status is "On". The object (principal) ID is "6d272fe3-0d59-47ec-ab5a-7950beb15e1b". The permissions section shows "Azure role assignments". A message at the bottom states: "This resource is registered with Microsoft Entra ID. The managed identity can be configured to allow access to other resources. Be careful when making changes to the access settings for the managed identity because it can result in failures. [Learn more](#)".

Configuración de la web app

Microsoft Azure portal showing the configuration of a storage account. The page title is "storageesemana14 | Storage browser". The left sidebar shows the navigation menu with "Storage browser" selected. The main content area shows the "Blob containers" section. The authentication method is "Access key". The search filter is "Search blobs by prefix (case-sensitive)". The table shows 1 item:

Name	Last modified	Access tier	Blob type	Size	Lease state
HelloWorld	7/10/2026, 9:27:13 a. m.	Hot (Inferred)	Block blob	60 B	Available

Creación del hello world (indicando que si funciona el service connection)



TU ESPACIO DE ARCHIVOS

Sube tu archivo

Selecciona un archivo de tu dispositivo y guárdalo en la nube.

El archivo «Screenshot 2026-10-07 174904.png» se subió correctamente.

Seleccionar archivo

Cualquier tipo de archivo - Máximo 20 MB

No se ha seleccionado ningún archivo

Ningún archivo seleccionado.

Se conservará el nombre original. Si ya existe, deberás renombrar tu archivo para subirlo.

Si ocurre un error, vuelve a seleccionar el archivo para intentarlo de nuevo.

App finalizada y en funcionamiento (se subió una imagen como efectos de prueba)

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

edsantisteban@correo...
UNIVERSIDAD RAFAEL LANDIVAR...

Home > storagesemana14

storagesemana14 | Storage browser

Storage account

Help me save costs by tiering unused blobs

Search

- Overview
- Activity log
- Tags
- Diagnose and solve problems
- Access Control (IAM)
- Data migration
- Events
- Storage browser**
- Storage Mover
- Partner solutions
- Resource visualizer
- Data storage
- Security + networking
- Data management
- Settings
 - Configuration
 - Data Lake Gen2 upgrade
 - Resource sharing (CORS)

storagesemana14

- Favorites
- Recently viewed
- Blob containers
 - \$logs
 - default
- View all
- Classic file shares
- Queues
- Tables

+ Add Directory | Upload | Change access level | Refresh | Delete | Copy | Paste | Rename | Acquire lease

Blob containers > default

Authentication method: Access key (Switch to Microsoft Entra user account)

Add filter

Search blobs by prefix (case-sensitive)

Only show active blobs

Showing all 2 items

	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	HelloWorld	7/10/2026, 5:20:37 p. m.	Hot (Inferred)	Block blob	60 B	Available
☐	Screenshot 2026-10-...	7/10/2026, 5:50:50 p. m.	Hot (Inferred)	Block blob	193.92 KiB	Available

Verificación de funcionamiento de la app (la imagen subida aparece en el container designado)